

Security Review

2026-09-22 by SECBIT Labs

At the request of the AladdinDAO team, we conducted a security review of changes to the f(x) Protocol's ezETH market contracts.

The change can be found at: <https://github.com/AladdinDAO/aladdin-v3-contracts/pull/275>

Initial review commit: `d5049895ea34a4b3782203c5a32f3ed953b8d032`.

Final review commit: `2c0b9e589063524cf127b1e95f3d6e04cd3b9e38`.

This report covers the background, implementation details, analysis, potential issues, and security review conclusions.

PR #275: ezETH Market Wind-Down

Change Background

ezETH now represents only a small proportion of fxUSD's underlying asset composition. In consideration of market support and protocol operations, the f(x) Protocol has decided to discontinue support for ezETH as an underlying protocol asset, distribute the remaining ezETH to fezETH and xezETH holders in accordance with a predetermined allocation plan, and ultimately shut down the ezETH market.

Change Details

The changes apply exclusively to the ezETH market and include the following:

1. Discontinuation of new ezETH market positions

The new Treasury implementation disables the fezETH and xezETH minting interfaces, preventing users from creating new fToken or xToken positions with ezETH. It also prevents the Treasury from performing operations used during normal protocol operation, including settlement, transfers of funds to strategies, and yield harvesting.

2. Fixed wind-down allocation for fezETH and xezETH

During wind-down initialization, the contract records:

- The Treasury's current ezETH balance;
- The total supplies of fezETH and xezETH;
- The asset allocation weights assigned to fezETH and xezETH.

Based on these snapshots, the contract calculates the respective amounts of ezETH allocated to fezETH and xezETH holders. Subsequent redemptions are executed at fixed exchange ratios rather than being calculated dynamically using the real-time collateral ratio and oracle price.

3. Closure of the related Rebalance Pools

The new Rebalance Pool implementation prevents users from continuing to invoke operations such as `deposit()`, `withdraw()`, `withdrawFrom()`, and the standard `liquidate()` function.

The remaining fezETH held by the Pools is collectively redeemed for ezETH by the administrator through `windDown()`. The resulting ezETH is then distributed to holders of pool shares through the reward accumulation mechanism. Users may subsequently claim their ezETH through the existing reward-claiming interfaces.

4. Removal of the ezETH market from fxUSD

The `FxUSD` contract introduces a new `removeMarket()` function. Once the amount of managed fToken recorded for the market has been reduced to zero, the administrator may:

- Remove ezETH from the list of supported markets;
- Revoke the allowance granted by FxUSD to the ezETH Market;
- Clear the market configuration;
- Transfer any residual fezETH dust held by the FxUSD contract to the administrator.

The corresponding ezETH Rebalance Pools will also be removed from the list of Pools supported by fxUSD. During the migration, the rUSD exposure previously backed by ezETH will be replaced with assets from other markets.

Change Analysis

The changes primarily involve two contracts designed specifically for the ezETH market wind-down:

`WrappedTokenTreasuryV2WindDown`, which handles the underlying redemption of fezETH and xezETH, and `FxUSDShareableRebalancePoolWindDown`, which manages the fezETH held on behalf of users.

1. WrappedTokenTreasuryV2WindDown Contract

This contract inherits from `WrappedTokenTreasuryV2` and serves as the dedicated wind-down implementation for the ezETH Treasury.

To prevent the system from creating new positions or executing normal operational logic, the contract overrides and disables several interfaces, including `mintFToken()` and `mintXToken()`. Calls to these disabled functions revert with `ErrorWindDownNotAllowed`.

Subsequent commits also override `maxRedeemableFToken()`, `maxRedeemableXToken()`, and `isUnderCollateral()`. These changes are intended to remove the wind-down redemption path's dependency on the ezETH oracle, which is no longer being updated, and to prevent fxUSD from treating the market as undercollateralized.

`initializeWindDown()` initializes the irreversible wind-down parameters. It verifies that the actual ezETH balance and the total supplies of fezETH and xezETH match the expected values submitted by the administrator. It then uses `fweight` and `xweight` to calculate the amount of ezETH allocated to each tranche.

Most importantly, the `redeem()` function has been restructured to calculate `_baseTokenOut` using the fixed supply and asset allocation recorded during initialization. Once the public redemption period has ended, the administrator may call `finalizeWindDown()` to close redemptions and subsequently use `adminClaim()` to withdraw any ezETH remaining in the Treasury.

2. FxUSDShareableRebalancePoolWindDown Contract

This contract inherits from `ShareableRebalancePool` and handles the remaining fezETH held by the ezETH-related Rebalance Pools.

After entering wind-down mode, the contract overrides and disables functions including `deposit()`, `withdraw()`, `withdrawFrom()`, and `liquidate()`. Users can no longer deposit or directly withdraw fezETH. Instead, the Pool's underlying assets are collectively converted into ezETH rewards through `windDown()`.

Once the Treasury wind-down has been finalized, the administrator may also use the Pool's `adminClaim()` function to withdraw the contract's remaining active and historical reward tokens.

Potential Issues Found

1. Dependency on an Inactive Oracle Prevents Wind-down Redemptions

In the initial implementation, although the Treasury's actual payout was derived from a fixed snapshot, users were still required to complete redemptions through the unchanged `MarketV2` contract.

Before calling the new Treasury's `redeem()` function, `MarketV2.redeemFToken()` and `MarketV2.redeemXToken()` first call `maxRedeemableFToken()` and `maxRedeemableXToken()`. The initial implementation directly inherited the original Treasury logic, under which these functions eventually call `_fetchBaseTokenPrice()` and read the ezETH oracle.

Because the on-chain ezETH price feed provided by RedStone is no longer being updated, the oracle call may revert and prevent the following operations from being completed:

- Users redeeming fezETH through `redeemFToken()`;
- Users redeeming xezETH through `redeemXToken()`;
- Rebalance Pools redeeming their fezETH holdings through `windDown()`;
- fxUSD redeeming fezETH through the original Market interface to complete the migration of rUSD's underlying assets.

The original failure paths included:

- [FxUSDShareableRebalancePoolWindDown.windDown\(\). calling redeemFToken\(\).](#)
- [TreasuryV2._fetchBaseTokenPrice\(\). validating the oracle price](#)

In a subsequent commit, the developers overrode `maxRedeemableFToken()`, `maxRedeemableXToken()`, and `isUnderCollateral()`, thereby removing the dependency on the ezETH oracle from the primary wind-down execution path.

It should be noted that if `xTokenRedeemPausedInStabilityMode` is enabled in the future, `MarketV2.redeemXToken()` may still call the inherited `collateralRatio()` function. Therefore, this setting should remain disabled throughout the wind-down.

2. Redundant fToken Approval in `windDown()`

In the initial implementation, `windDown()` executed the following approvals before calling `MarketV2.redeemFToken()`:

```
safeApprove(market, 0);
safeApprove(market, liquidated);
```

However, `MarketV2.redeemFToken()` does not transfer fToken into the Market through `transferFrom()`. Instead, it calls the Treasury's `redeem()` function, and the Treasury directly burns the corresponding amount of fToken from the `_owner` address.

The approval is therefore unnecessary for the redemption process and results in additional gas consumption and unnecessary allowance state changes.

The developers removed this approval logic in a subsequent commit and now call `MarketV2.redeemFToken()` directly to complete the redemption.

3. Absence of On-chain Liability Settlement Constraints for Finalization and Reward Recovery

`finalizeWindDown()` does not verify whether:

- All fezETH and xezETH have been redeemed;
- Both relevant Rebalance Pools have executed `windDown()`;
- The required minimum claiming period has elapsed;
- Any unsettled user liabilities remain.

The administrator could therefore theoretically call `finalizeWindDown()` immediately after wind-down initialization and subsequently withdraw the Treasury's remaining ezETH through `adminClaim()`.

Once the Treasury has been marked as `Finalized`:

- Unredeemed fezETH and xezETH can no longer be redeemed because the operation reverts with `ErrorWindDownFinalized`;
- A Pool that has not yet executed `windDown()` can no longer redeem its fezETH holdings because the Pool requires the Treasury status to remain `WindDown`;
- The Pool's standard withdrawal interfaces have already been disabled, potentially leaving its fezETH holdings locked.

In addition, the Pool's `adminClaim()` function iterates through all active and historical reward tokens and transfers the contract's entire balance of each token to the administrator. The function does not verify whether:

- Any user rewards remain claimable;
- Any rewards remain pending linear distribution;
- All Pool users have completed their claims;
- The withdrawable balance consists solely of rounding dust.

If the administrator invokes this function prematurely, users who have not yet claimed rewards denominated in ezETH, FXN or other tokens may be unable to complete their claims because the Pool no longer holds sufficient balances.

These issues do not constitute a privilege-escalation vulnerability exploitable by ordinary users, as the relevant operations are restricted to the administrator, who already holds proxy upgrade authority. Nevertheless, they increase the risk of operational errors and incorrect execution sequencing. Provided that the ezETH market is being closed and the administrator strictly follows the prescribed procedure, these risks can be controlled through operational processes.

4. Returning Zero as the Maximum Redeemable Amount Selects the Higher Fee Tier

The new implementation sets the second return value of both `maxRedeemableFToken()` and `maxRedeemableXToken()` to `0`.

In the original implementation:

- The second return value of `maxRedeemableFToken()` represents the maximum amount of fToken that may be redeemed before the system's collateral ratio rises to the specified target;
- The second return value of `maxRedeemableXToken()` represents the maximum amount of xToken that may be redeemed before the system's collateral ratio falls to the specified target.

`MarketV2` uses these values as thresholds for its tiered fee calculation. For any non-zero redemption amount, setting the threshold to `0` causes the fee calculation to select the second fee tier. Based on the current on-chain parameters:

- The fezETH redemption fee would be `0.25%`;
- The xezETH redemption fee would be `9%`.

If a user specifies a sufficiently low `_minBaseOut`, the fee will be deducted from the redemption proceeds. If `_minBaseOut` is set using the fee-free wind-down preview, the transaction will instead revert because the final output is lower than the specified minimum.

From the perspective of wind-down semantics, a more appropriate return value for these functions would be:

```
return (0, type(uint256).max);
```

Keeping the first return value at `0` prevents the original Market's additional bonus logic from being triggered. Setting the second return value to `type(uint256).max` removes the xToken redemption cap imposed by this value, but does not bypass the separate `collateralRatio()` check when `xTokenRedeemPausedInStabilityMode` is enabled.

However, changing only the second return value to `type(uint256).max` would still not fully satisfy the zero-fee requirement:

- fezETH redemptions would use the first fee tier, which is currently `0%`;
- xezETH redemptions would use the first fee tier, which is currently `2%`.

Accordingly, before any user or Pool redemption is executed, the actual upgrade process must also call:

```
MarketV2(EZ_MARKET).updateRedeemFeeRatio(0, 0, true);
MarketV2(EZ_MARKET).updateRedeemFeeRatio(0, 0, false);
```

The current Foundry test includes both configuration operations. Provided that both redemption fee configurations have been set to zero and `xTokenRedeemPausedInStabilityMode` remains disabled, retaining `0` as the second return value will not affect the final fee charged.

Review Results

After reviewing the development team's ezETH market wind-down implementation, subsequent remediations, and supporting Foundry validation, and discussing the identified issues with the team, we consider it reasonable to proceed with merging the reviewed code. This assessment is conditional on strict adherence to the agreed execution plan, including setting both fezETH and xezETH redemption fees to zero before any redemption, keeping `xTokenRedeemPausedInStabilityMode` disabled, completing the rUSD backing migration and all Rebalance Pool wind-down operations before finalization, and refraining from invoking `adminClaim()` until all user liabilities and reward claims have been settled. Provided these conditions are met and the migration is continuously monitored, we expect the implementation and residual operational risks to remain within acceptable bounds.